

Lab 1: Python Basics, File Handling, NumPy, and Pandas

 **Duration: 1 Hour**

 **Objective:**

This lab is designed to assess and reinforce students' understanding of **Python basics, file handling, NumPy, and Pandas**. Students must **complete missing code** in the given blocks and verify their solutions using the provided **test functions**.

Part 1: Python Basics - Variables, Functions, and Loops

1 Define a Function to Calculate Factorial

Fill in the missing code to compute the **factorial** of a given number using a loop.

```
In [1]: def factorial(n):
        """Compute the factorial of a number."""
        result = 1
        for i in range(1, n + 1):
            result *= i
        return result

#  Test Case
def test_factorial():
    assert factorial(5) == 120, "Test case failed!"
    assert factorial(3) == 6, "Test case failed!"
    assert factorial(1) == 1, "Test case failed!"
    print( All test cases passed!")

test_factorial()
```

 All test cases passed!

2 Reverse a String

Complete the function to reverse a given string.

```
In [2]: def reverse_string(s):
        """Reverse a string."""
        return s[::-1]
```

```
#  Test Case
def test_reverse_string():
    assert reverse_string("Python") == "nohtyP", "Test case failed!"
    assert reverse_string("Data") == "ataD", "Test case failed!"
    print(" All test cases passed!")

test_reverse_string()
```

 All test cases passed!

3 Find Even Numbers in a List

Complete the function to return a list of even numbers from the given list.

In [3]:

```
def find_even_numbers(lst):
    """Return a list of even numbers."""
    return [num for num in lst if num % 2 == 0]

#  Test Case
def test_find_even_numbers():
    assert find_even_numbers([1, 2, 3, 4, 5, 6]) == [2, 4, 6], "Test case failed!"
    assert find_even_numbers([10, 15, 20, 25]) == [10, 20], "Test case failed!"
    print(" All test cases passed!")

test_find_even_numbers()
```

 All test cases passed!

Part 2: File Handling

4 Writing and Reading a File

Fill in the missing parts to **write a list of names to a file** and **read the content**.

In [4]:

```
# Writing to a file
def write_to_file(filename, data):
    with open(filename, 'w') as file:
        for name in data:
            file.write(f"{name}\n")

# Reading from a file
def read_from_file(filename):
    with open(filename, 'r') as file:
        return file.readlines()

#  Test Case
def test_file_handling():
    filename = "names.txt"
```

```

names = ["Alice", "Bob", "Charlie"]

write_to_file(filename, names)
content = read_from_file(filename)

assert content == ["Alice\n", "Bob\n", "Charlie\n"], "Test case failed!"
print("✅ All test cases passed!")

test_file_handling()

```

✅ All test cases passed!

5 Writing and Reading a JSON File

Fill in the missing parts to write and read a dictionary from a JSON file.

In [5]:

```

import json

# Writing to a JSON file
def write_json(filename, data):
    with open(filename, 'w') as file:
        json.dump(data, file, indent=4)

# Reading from a JSON file
def read_json(filename):
    with open(filename, 'r') as file:
        return json.load(file)

# ✅ Test Case
def test_json_handling():
    filename = "data.json"
    data = {"name": "Alice", "age": 25, "city": "Paris"}

    write_json(filename, data)
    loaded_data = read_json(filename)

    assert loaded_data == data, "Test case failed!"
    print("✅ All test cases passed!")

test_json_handling()

```

✅ All test cases passed!